

Introduzione sull'UML

lunedì 3 marzo 2025 14:27

UML (Unified Modeling Language) è **linguaggio di modellazione** basato su modelli **orientati a oggetti (Object-Oriented)**

L'UML definisce **14 tipi di diagrammi** per modellare un'applicazione sotto prospettive diverse.

I principali tipi di diagrammi sono:

- **Diagrammi strutturali:**
 - Diagramma delle **classi e oggetti** (class and object diagram)
- **Diagrammi comportamentali:**
 - Diagramma degli **use case** (use case diagram)
 - Diagramma degli **stati e transizioni** (state/transition diagram)
 - **Interaction** (Sequence e Collaboration diagram)
 - **Activity diagram**
- **Diagrammi architetturali:**
 - **Component** diagram
 - **Deployment** diagram

Il corso si concentra sugli aspetti base e sui diagrammi più importanti, e verranno usati solo questi diagrammi (e solo le caratteristiche più importanti):

- **Diagrammi strutturali:**
 - Diagramma delle **classi e oggetti** (class and object diagram)
- **Diagrammi comportamentali:**
 - Diagramma degli **use case** (use case diagram)
 - Diagramma degli **stati e transizioni** (state/transition diagram)

Obiettivi

Per **progettare applicazioni** di base di dati reali e di grandi dimensioni è necessario spendere gran parte del tempo per **comprendere** i dati di interesse e le loro **interrelazioni**. Non esiste una soluzione unica o una ricetta base da applicare ma bisogna fare delle **scelte ragionate**. È necessario quindi:

- Ragionare logicamente
- Considerare le conseguenze delle nostre scelte
- Valutare le alternative
- Scomporre i problemi complessi in sottoproblemi

Esempio

Se dovessimo creare un'applicazione che permetta di mantenere informazioni su un insieme di contatti (telefonici e mail) è facile definire uno schema di relazione per la base dei dati. I requisiti richiedono di mantenere il nome, cognome, numero e indirizzo dei contatti e permettere all'utente di effettuare delle azioni per la creazione/modifica/cancellazione/ricerca di un contatto o aggiungerli/rimuoverli da un gruppo.

Se invece si vuole sviluppare un sistema che permette ad una banca di gestire i conti correnti dei clienti, i loro investimenti e la propria rete di promotori finanziari, allora la creazione di uno **schema di relazione** risulta **complesso**.

Il sistema deve **tenere traccia** di tutti gli acquisti e vendite di azioni, obbligazioni, ecc. effettuati dai clienti, e poter calcolare la valorizzazione del loro portafoglio. Inoltre deve assistere i promotori finanziari nella scelta degli strumenti finanziari più adeguati da proporre ai clienti, e permettere ai responsabili di agenzia di controllare la professionalità dei promotori.

È **impossibile** scrivere **direttamente** lo **schema relazionale** e il **programma** per l'applicazione.

Progetto software complesso:

- pool di Ingegneri del SW, progettisti, programmatori: 1.5 anni (spannometricamente...)
- tempo per capire il problema: 6 mesi (33%)
- tempo per la progettazione: 9 mesi (50%)
- tempo per la realizzazione: 3 mesi (solo il 17%)
- più: tempo per test & verifica, ecc

Contesto Organizzativo

Bisogna essere in grado di distinguere le **figure o i ruoli** coinvolti nelle varie fasi del **ciclo di vita del software (progettazione, sviluppo ed esercizio)**:

- **Committente:** Chi finanzia e commissiona il progetto, definendo requisiti e obiettivi
- **Esperti del dominio:** Specialisti del settore di applicazione del software, forniscono conoscenze e specifiche
- **Analisti:** Raccolgono e analizzano i requisiti, traducendoli in specifiche tecniche e funzionali
- **Progettisti:** Definiscono l'architettura e la struttura del software
- **Programmatori:** Scrivono il codice sulla base delle specifiche fornite
- **Utenti finali:** Coloro che utilizzeranno il software e ne valuteranno l'**usabilità**
- **Manutentori:** Si occupano di aggiornamenti, correzione bug e miglioramenti post-rilascio

Esempio

Il Comune di XYZ vuole automatizzare la gestione delle info relative alle contravvenzioni elevate sul suo territorio, in particolare vuole dotare ogni vigile di una app per smartphone che gli consenta di comunicare al sistema informatico il veicolo, il luogo e la natura della contravvenzione.

Il sistema informatico notificherà, tramite posta ordinaria, la contravvenzione al cittadino interessato.

Il Comune bandisce gare per la progettazione, realizzazione e manutenzione del sistema, che vengono vinte da ITSolutions s.p.a, Develop s.r.l. e Ops s.r.l.

Quali sono i ruoli coinvolti in questa progettazione del software?

- **Committente:** Comune di XYZ
- **Esperti del dominio:** Funzionario del Comune o altro professionista designato, esperto del Codice della Strada
- **Analisti:** Personale ITSolutions s.p.a.
- **Progettisti:** Personale ITSolutions s.p.a.
- **Programmatori:** Personale Develop s.r.l.
- **Utenti finali:** Polizia locale XYZ
- **Manutentori:** Personale Ops s.r.l.

Ciclo di Vita del Software

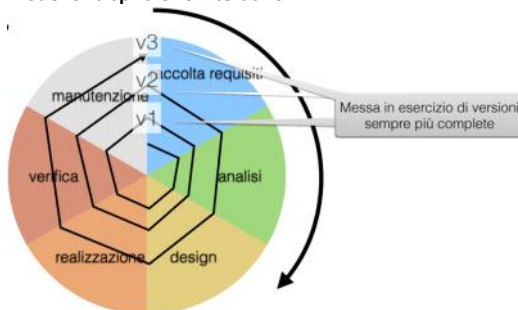
Macro-fasi Principali

1. **Studio di fattibilità**
 - Comprendere i **requisiti di alto livello**
 - Valutare **costi e benefici**
 - Pianificare le **attività** e le **risorse** del progetto
 - Individuare l'**ambiente di programmazione** (hardware/software)
2. **Raccolta dei requisiti**
 - Raccolta dei **requisiti** presso i diversi ruoli
 - Stesura e sintesi iniziali
 - **Raffinamento** dei requisiti
3. **Analisi concettuale dei requisiti**
 - **Obiettivo**: produrre lo **schema concettuale** dell'applicazione, che definisce in dettaglio **cosa** l'applicazione dovrà realizzare, indipendentemente dal **come**.
 - Lo **schema concettuale**:
 - Modella i **dati di interesse**, le loro **articolazioni**, **interrelazioni ed evoluzioni possibili**
 - Specifica i **servizi computazionali** che l'applicazione dovrà offrire ai diversi utenti
 - Lo schema concettuale è un modello logico-matematico dell'applicazione, e sarà la base da cui partire per le successive attività di progettazione
4. **Progetto (design) dell'applicazione**
 - **Obiettivo**: specificare **come** l'applicazione dovrà realizzare le sue funzioni (ora che il **cosa** è stato definito nel punto 3)
 - Scegliere il mix tecnologico ottimale per l'applicazione
 - Definire l'**architettura**
 - Definire le strutture di **rappresentazione dei dati** (in memoria centrale e di massa)
5. **Realizzazione (implementazione) dell'applicazione**
 - Scrivere il **codice**
 - Scrivere la **documentazione**
6. **Integrazione dei componenti e verifica dell'applicazione**
 - Le diverse componenti dell'applicazione, sviluppate separatamente, **vengono integrate**
 - Si valuta se l'applicazione svolge correttamente, completamente ed efficientemente i suoi compiti
7. **Messa in esercizio dell'applicazione**
 - L'applicazione viene **messa in esercizio** ed inizia a funzionare
8. **Manutenzione dell'applicazione**
 - L'applicazione viene **monitorata** durante l'esercizio
 - Correzioni ed aggiornamenti vengono prodotti ove e quando necessario

Al termine di ogni fase, se necessario, si può **tornare indietro**

Modelli

- **Modello "a cascata" ("waterfall model")**
 - Ogni attività inizia quando termina la precedente
 - Ha un interesse esclusivamente didattico
- **Modello "a spirale" o "iterativo"**



Classi, Oggetti e Link

lunedì 3 marzo 2025 16:36

Nella fase di **analisi** (punto 3 del ciclo di vita del software) ci si **concentra di più sulle classi** che sugli **oggetti**. Questi ultimi servono principalmente per descrivere elem. singoli particolarmente significativi oppure per descrivere esempi.

Oggetti in UML

In UML un oggetto modella un elem. del dominio di analisi che:

- ha "vita propria"
- è **identificato univocamente** dall'identificatore di oggetto
- è **istanza di una classe**
- in alcuni casi di più classi, ma comunque tra le classi di cui un oggetto è istanza, esiste sempre la classe più specifica

Esempio

- **div_comm** è l'ID di oggetto
- **Libro** è la classe più specifica di cui l'oggetto è istanza
- L'oggetto si segna con la **sottolineatura**



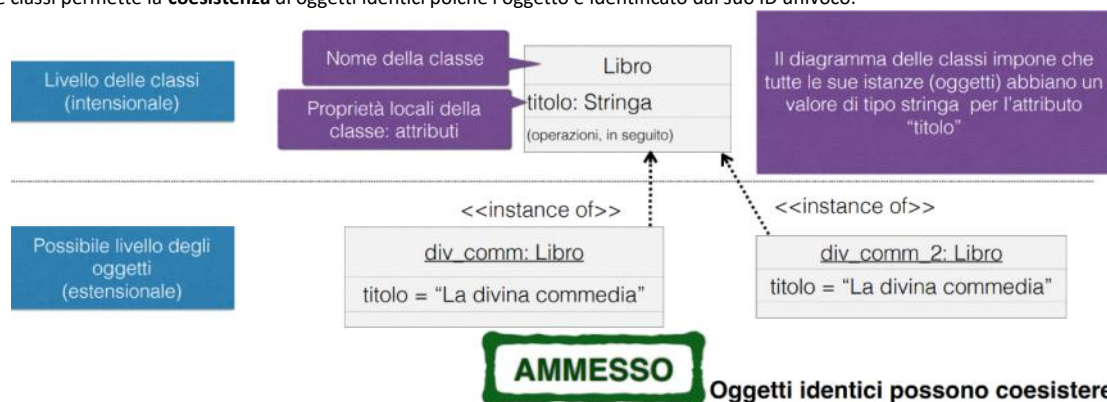
Classi

Una **classe** modella un insieme di **oggetti omogenei** (le istanze della classe) ai quali sono associate **proprietà statiche (attributi)** e **dinamiche (operazioni)**

Ogni classe è descritta da:

- **Un nome**
- **Un insieme di proprietà statiche o dinamiche** (astrazioni delle proprietà degli oggetti che sono istanze delle classi)

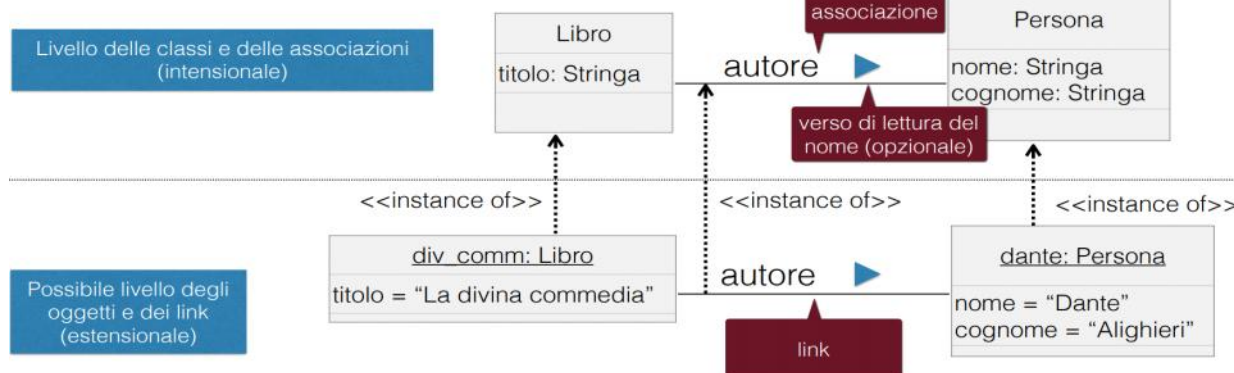
Il diagramma delle classi permette la **coesistenza** di oggetti identici poiché l'oggetto è identificato dal suo ID univoco.



Link e Associazioni

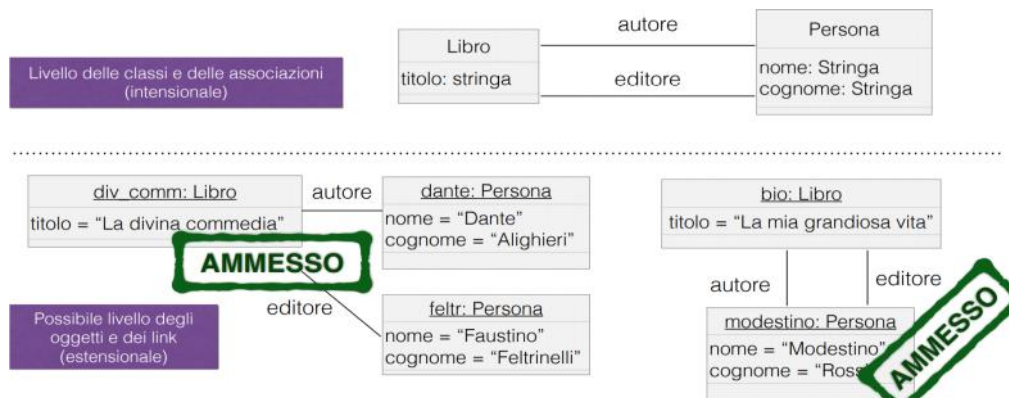
Una **associazione** modella la possibilità che gli oggetti di più classi abbiano **dei legami**. Le **istanze di associazione** si chiamano **link**.

Es. Se A è una associazione tra le classi C1 e C2, una istanza di A è un link tra due oggetti (quindi una coppia), uno della classe C1 e l'altro della classe C2



I link quindi sono **istanze delle associazioni**. Quindi se l'associazione è un **insieme di coppie** (Es. Libro - Persona), il link è una di quelle coppie (es. div_comm - Dante). Però al contrario degli oggetti, i **link non hanno ID univoci**, ma sono **implicitamente identificati** dalla coppia di oggetti che rappresenta. Quindi non è ammesso (più che ammesso, non ha concettualmente senso) usare più di un link per collegare due istanze

Tra le stesse classi possono essere definite **più associazioni**, che modellano **legami di natura diversa**



Vediamo come Libro e Persona hanno due associazioni, però nel livello degli oggetti **non è obbligatorio** usare entrambi i link per definire l'autore e l'editore (si può anche usare un solo link ad esempio per l'autore)

Association Class

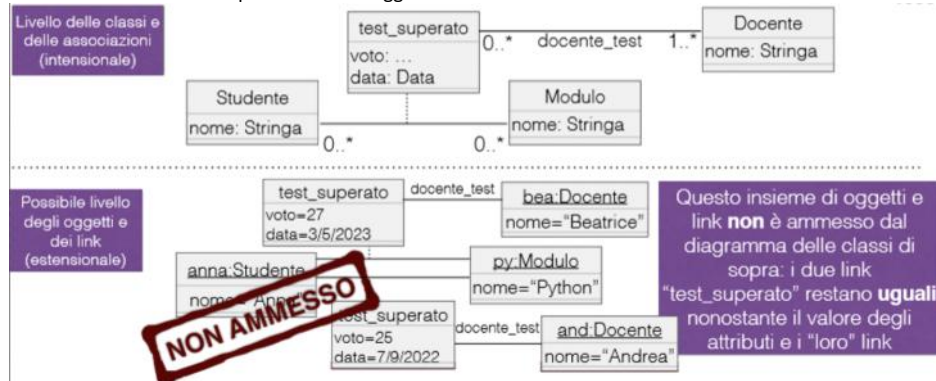
Se non possiamo aggiungere un attributo intrinseco ad una coppia di classi associate allora è probabile che quell'attributo sia una proprietà dell'associazione stessa. Si può quindi definire una **associazione con attributi (association class)** a cui diamo gli attributi necessari intrinseci all'associazione.

Ad es. se un sistema deve gestire gli esiti (voto e data) dei moduli superati dagli studenti, allora **voto** non è proprietà locale né di Studente, né di Modulo, ma è una proprietà del **legame** dell'oggetto Studente e Modulo.

E poiché l'associazione con attributi è essa stessa una **classe** possiamo collegarla ad altre classi e può essere **superclasse/sottoclasse** di un'altra classe.

Ad esempio il test_superato si può collegare col Docente con cui si è effettuato il test

Ovviamente, come prima, **non sono ammessi** dall'UML più link tra due oggetti

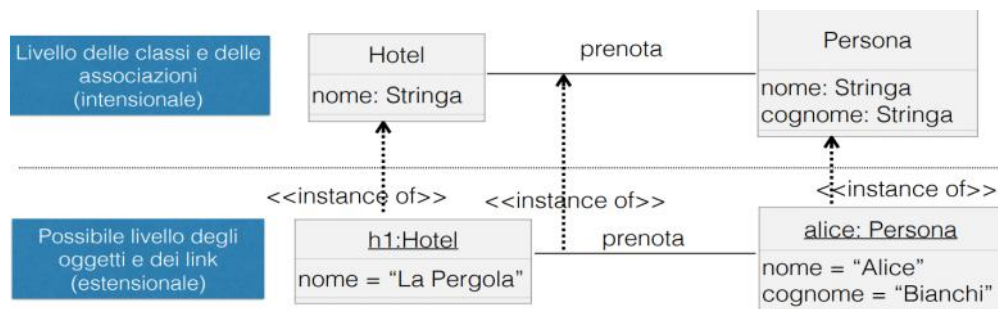


L'associazione con attributi inizia con la **lettera minuscola** ed ha una **linea tratteggiata** che la lega con l'associazione tra le classi.

Esempio

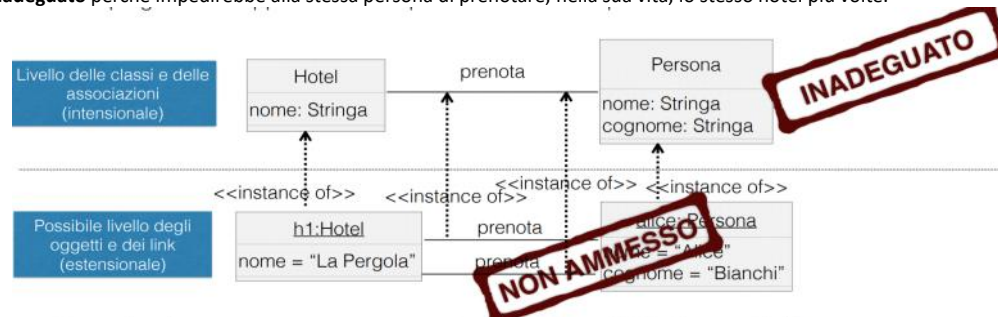
Si vuole progettare un'applicazione che permette ai clienti di prenotare l'hotel via web.

Esempio 1 delle classi



Però se provassimo a rappresentare una seconda prenotazione di "alice" presso "h1" non potremmo perché non è ammesso.

Quindi il diagramma è **inadeguato** perché impedirebbe alla stessa persona di prenotare, nella sua vita, lo stesso hotel più volte.



Possiamo risolvere inserendo la classe **Prenotazione** permettendo ad ogni singola prenotazione (oggetto) di avere "vita propria"



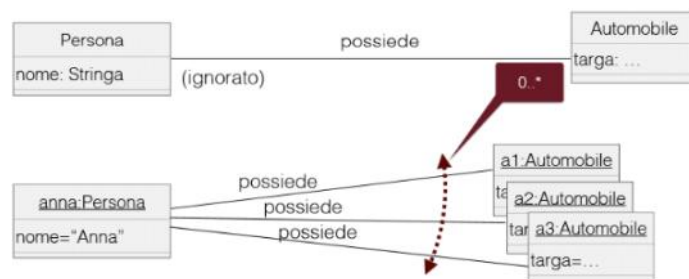
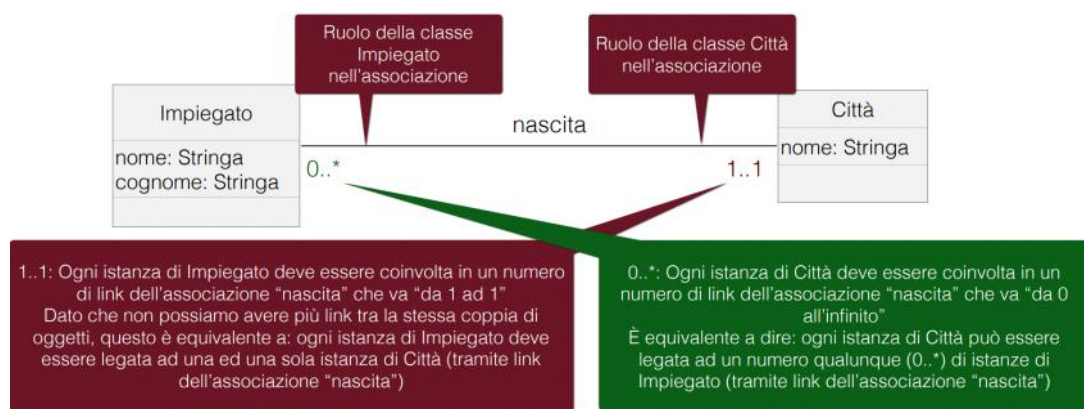
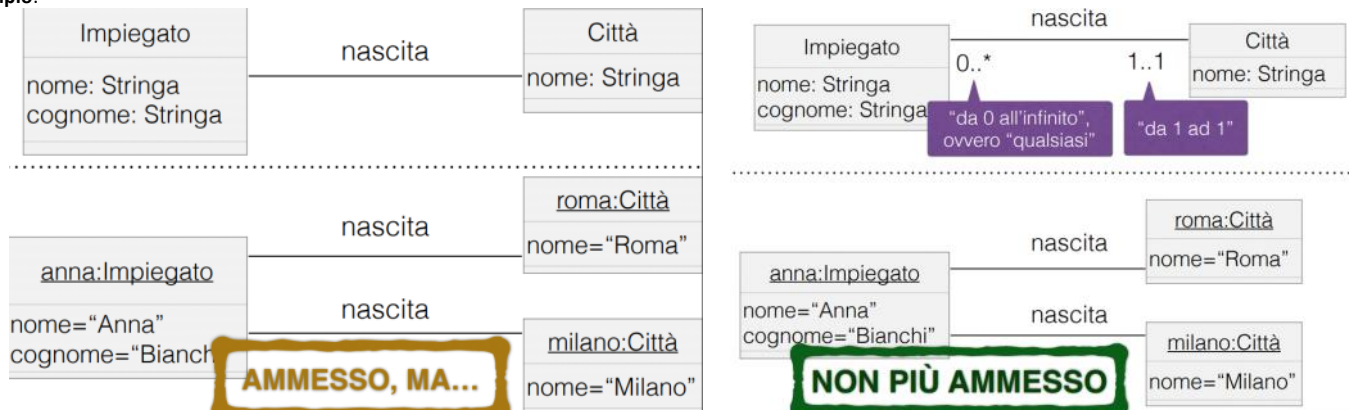
Vincoli di molteplicità

lunedì 3 marzo 2025 17:46

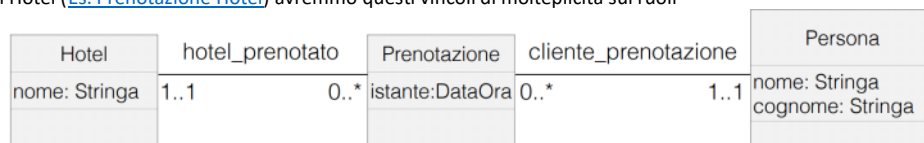
Per soddisfare i **vincoli** del mondo reale bisogna imporre dei **vincoli di molteplicità sulle associazioni e sugli attributi**.

UML permette di definire sul diagramma delle classi dei **vincoli di integrità**, che impongono ulteriori **restrizioni** sui livelli estensionali ammessi.

Esempio:



Ad esempio sull'esercizio sull'Hotel ([Es. Prenotazione Hotel](#)) avremmo questi vincoli di molteplicità sui ruoli

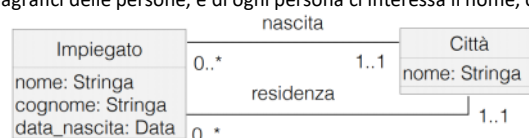


DA IMPARARE A MEMORIA (perché il prof senno ti mena):

- "A quanti link hotel_prenotato ogni istanza di hotel può partecipare?"
- "Ogni istanza della classe Hotel a quanti link hotel_prenotato può partecipare?"
- "Un istanza di Hotel può partecipare da 0 a * link dell'associazione hotel_prenotato" oppure
- "Ogni istanza di Hotel deve essere coinvolta in un numero di link dell'associazione hotel_prenotato che va da 0 a *" e allo stesso modo
- "Un istanza di Prenotazione può partecipare ad esattamente 1 link dell'associazione hotel_prenotato" oppure
- "Ogni istanza di Prenotazione deve essere coinvolta in un numero di link dell'associazione hotel_prenotato che va da 1 a 1"

Esercizio

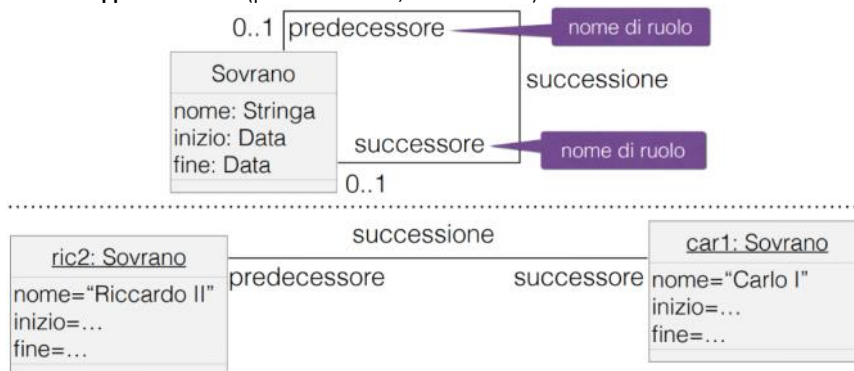
Si vuole progettare un sistema che gestisca i dati anagrafici delle persone, e di ogni persona ci interessa il nome, cognome, data di nascita e città di nascita e residenza



Associazioni con la classe stessa

Se volessimo modellare i sovrani di un regno per cui ci interessa il nome, il periodo in cui ha regnato e il suo **predecessore**, possiamo fare in modo che la classe sovrano stessa si associ con se stessa per indicare sia un **successore** e sia un **predecessore**.

Una istanza dell'associazione diventa una **coppia etichettata** (predecessore: s1, successore: s2)



Quando si usa un'associazione sulla classe stessa è **obbligatorio** inserire i nomi dei ruoli (es. predecessore/successore)

Tipi di Dati

martedì 4 marzo 2025 17:24

Nell'analisi bisogna scegliere dei dati che siano separati dalle scelte tecnologiche, quindi dobbiamo usare **tipi di dato concettuali**, che siano facilmente realizzabili con qualsiasi tecnologia informatica.

Tipi base

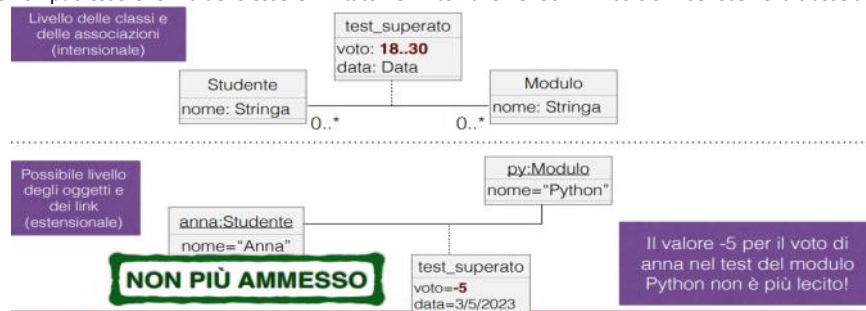
- **Intero** (1, 2, ...)
- **Reale** (1.15134, 15.51353, ecc)
- **Stringa** ("Anna", "Roma", ecc)
- **Booleano** (Vero, Falso)
- **Data** (5/10/2025)
- **Ora** (13:50)
- **DataOra** (5/10/2025 13:50)

Dati Specializzati

Per assicurarci che i valori siano leciti nel diagramma dobbiamo imporre dei **vincoli** sul valore dei dati, usiamo quindi dei **tipi di dati specializzati**:

Es: Intero > 0, Reale ≤ 0, ecc, tipo di intervallo (Intero): 0..100, 18..30, ecc

Ad esempio il voto di un esame non può essere -5 ma deve essere limitato nell'intervallo 18..30. Il vincolo si inserisce nella classe al posto del tipo dell'attributo



Tipi Enumerativi

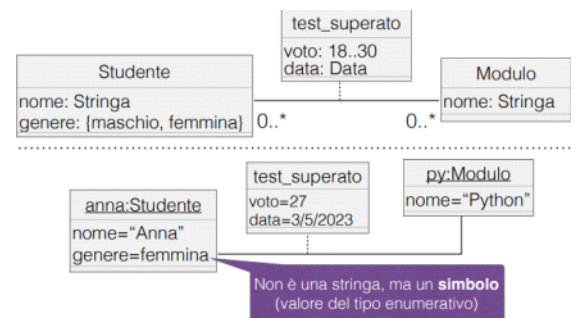
Se abbiamo un insieme di val. **finito e piccolo**, possiamo usare un **tipo di dato enumerativo** che definisce esplicitamente l'insieme dei val possibili per l'attributo.

Bisogna fare molta attenzione sull'uso del tipo enumerativo

poiché bisogna essere **sicuri** che i dati siano quelli e **non mutino nel tempo**.

Il dominio enumerativo si usa con le **parentesi graffe** e i val. sono **etichette** e non **stringhe**, quindi non bisogna usare le virgolette per indicare i valori.

Esempio: {maschio, femmina}, {Africa, America, Antartide, Asia, Europa, Oceania}, ecc



Dati definiti dall'Utente e Dati Composti

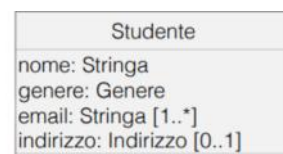
È permesso anche definire **nuovi tipi di dati**, nel caso ad esempio in cui un tipo di dato viene utilizzato spesso nelle classi.

È necessario definire il tipo di dato dandogli un nome e il suo valore.

Si possono inoltre usare **tipi di dato composti** da più campi, chiamati **tipi record** (o record)

Esempio:

- Tipo **Genere** = {maschio, femmina} (enumerativo)
- Tipo **Indirizzo** = (via: Stringa, civico: Intero > 0, CAP: Intero > 0) (record con 3 val)



Vincoli di Molteplicità sugli attributi

Si possono imporre dei vincoli di molteplicità anche sugli attributi stessi (il default 1..1 non viene scritto)

I vincoli di **molteplicità sugli attributi** sono segnati con le **parentesi quadre**.

Ad es.

- Ogni studente ha associato un solo nome e Genere
- Ogni studente ha associato **1 o più** email
- Ogni studente ha associato **al più un** indirizzo di casa



Vincoli di Identificazione di Classe

mercoledì 5 marzo 2025 12:24

Oltre ai:

- **Vincolo (di integrità):** affermazione che impone restrizioni all'insieme degli oggetti e link ammessi ulteriori a quelli strutturali dell'UML
- **Vincolo di molteplicità sulle associazioni**

È necessario imporre ulteriori vincoli per le classi, il **vincolo di identificazione di classe**

Esso impone che **non possono coesistere** oggetti di una classe che coincidono nel val. di un insieme di attributi e/o sono collegati tramite link agli stessi oggetti di altre classi.

Esempio:

- **Non possono esistere** due persone con lo **stesso CF** ("{id1}")
- **Non possono esistere** due persone con, simultaneamente, lo **stesso nome, cognome, data di nascita** ("{id2}")

Persona
cf: CodiceFiscale {id1}
nome: Stringa {id2}
cognome: Stringa {id2}
nascita: Data {id2}

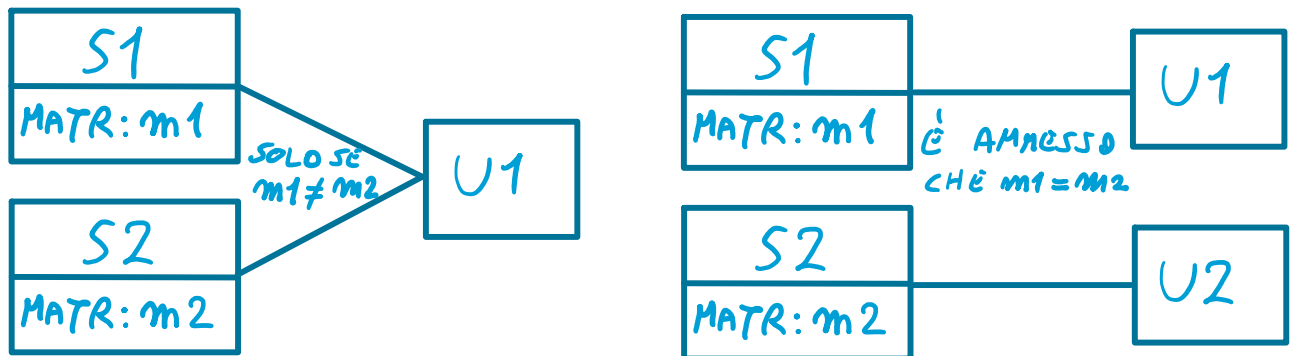
{id2} definisce un unico vincolo su tre attributi

Un **vincolo di identificazione di classe** può anche coinvolgere **ruoli della classe**

Esempio: Non possono esistere due studenti con la **stessa matricola nella stessa università**



ATTENZIONE: Il vincolo di identificazione di classe può coinvolgere solo **attributi a molteplicità [1..1]** e/o **ruoli della classe con molteplicità 1..1**



Generalizzazione

mercoledì 5 marzo 2025 12:43

Ereditarietà

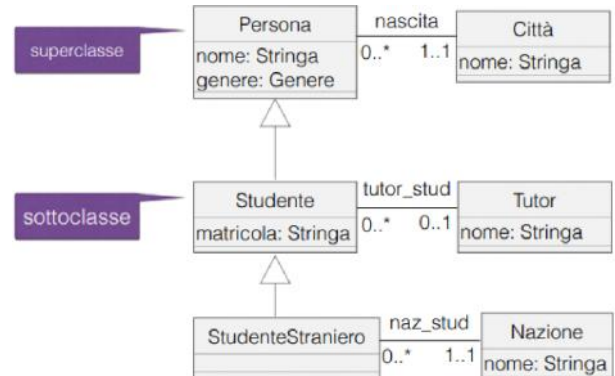
In molte situazioni si ha che **classi diverse** sussista una **relazione di sottoinsieme**. L'UML permette di definire il concetto di **relazione is-a tra classi**:

- **Relazione is-a ("è anche un")**: ogni istanza della classe **Studente** (**sotto-classe**) è concettualmente **anche** un'istanza della classe **Persona** (**super-classe**)
- Ovviamente **non vale il viceversa**: non tutte le istanze di **Persona** sono per forza istanze di **Studente**
- L'ereditarietà può essere anche a **più livelli (transitività)**, passando le classi associate e gli attributi alle sottoclassi.
- Una superclasse può avere **più sottoclassi**.

Esempio

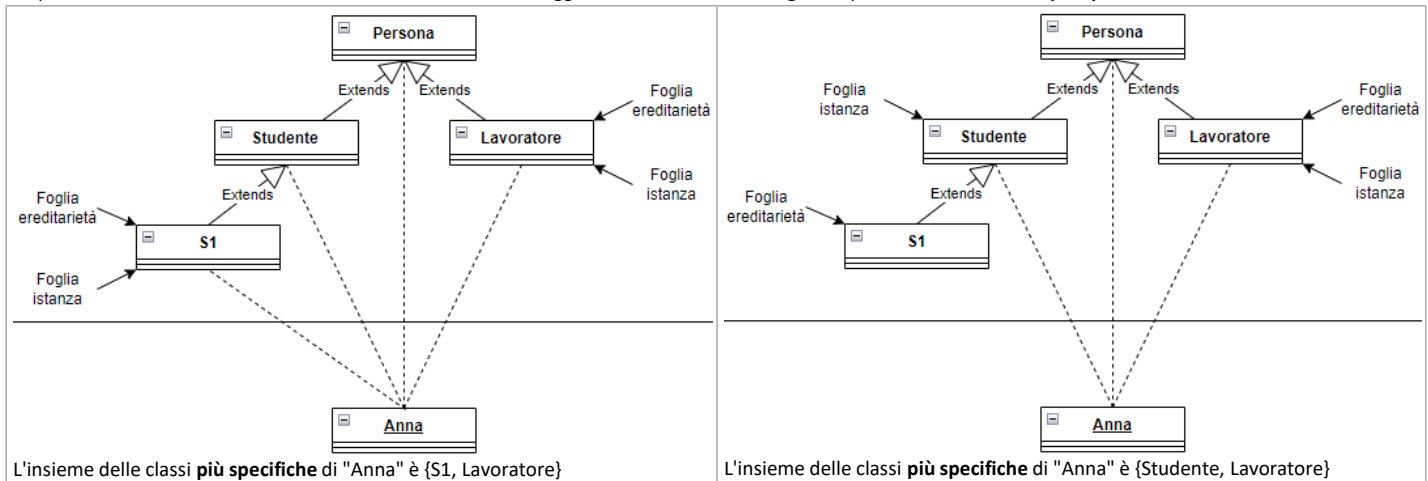
In presenza di relazioni is-a, vige il **meccanismo dell'ereditarietà**:

- Di tutte le **Persona** rappresentiamo nome, genere, città di nascita
- Di tutti gli **Studenti** rappresentiamo matricola e eventuale Tutor
- Di tutti gli **StudentiStranieri** rappresentiamo la loro Nazione d'origine
- Tutti gli **Studenti** sono **Person**e (relazione is-a) → **di conseguenza**, di tutti gli **Studenti** **rappresentiamo anche** nome, genere e città di nascita (con le loro molteplicità 1..1)
- Tutti gli **StudentiStranieri** sono **Studenti** e quindi **Person**e → **di conseguenza**, di tutti gli **StudentiStranieri** **rappresentiamo anche** nome, cognome, città di nascita e matricola



Classi più Specifiche

Le classi **più specifiche** di un oggetto sono un **insieme delle classi** di cui l'oggetto è istanza che **non sono a loro volta superclassi** di altre classi dell'oggetto. Se prendiamo l'albero di ereditarietà con solo le classi di cui l'oggetto è istanza allora le **foglie** di quell'albero sono **classi più specifiche**.



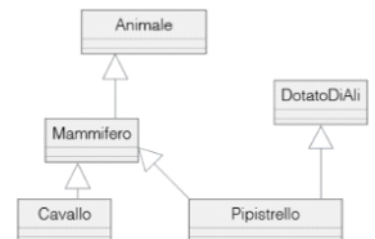
Dobbiamo contare solo l'albero di ereditarietà togliendo le classi di cui l'oggetto non è istanza (nel secondo esempio la classe S1).

Generalizzazione

La **generalizzazione** è un costrutto **più complesso** della relazione **is-a**, poiché permette di definire che gli oggetti possono essere istanze di **più classi figlie secondo uno stesso criterio concettuale**.

Esempio

- **Studente/Lavoratore is-a Persona**
- **Uomo/Donna is-a Persona**
- Secondo il **criterio del genere**, le **Person**e sono considerate **Uomini** e/o **Donne**
- Secondo il **criterio (indipendente) dell'occupazione**, le **Person**e sono considerate **Studenti** e/o **Lavoratori**
- È ancora possibile per un oggetto di essere istanza sia di **Lavoratore** che di **Studente** o entrambi
- Una classe può anche essere superclasse di **più generalizzazioni distinte (criteri diversi)**
- Una classe può essere **sottoclasse di più classi** avendo quindi una **ereditarietà multipla**



Il problema è che una istanza della classe base (**Persona**) può essere istanza di **più di una sottoclasse** di una **stessa generalizzazione** (sia **Uomo** che **Donna**).

Oppure una istanza della classe base (**Persona**) può essere istanza di **nessuna delle sottoclassi** di una **stessa generalizzazione** (né **Uomo** né **Donna**)

Bisogna quindi evitare questa situazione impostando dei **vincoli**.

Vincolo Disjoint

La generalizzazione **disgiunta non permette** ad un istanza di essere istanza di **entrambe** le sottoclassi.

Es:

Una istanza di **Persona** **non può più** essere istanza **sia di Uomo che di Donna**.

Quindi un istanza di **Uomo** **non può** essere anche istanza di **Donna**

Però un istanza di **Studente** **può** essere anche istanza di **Lavoratore**

Vincolo Complete

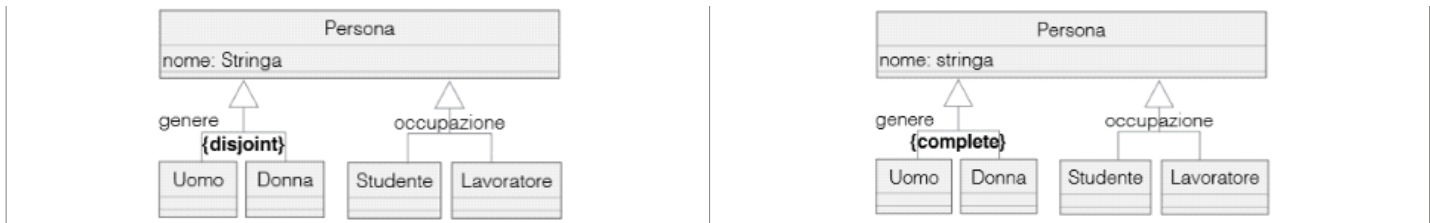
La generalizzazione **complete non permette** ad un istanza di non essere istanza di **nessuna** delle sottoclassi.

Es:

Una istanza di **Persona** **non può più** essere istanza **né di Uomo né di Donna**.

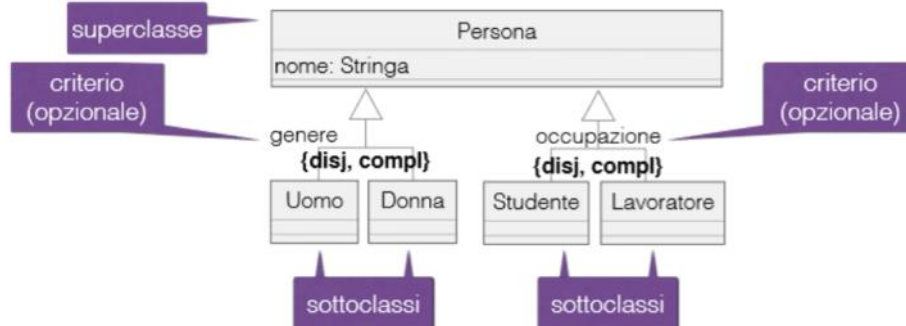
Quindi un istanza di **Persona** deve **essere per forza** anche istanza di **Uomo/Donna**

Però un istanza di **Persona** può **non essere** di **Studente** e **Lavoratore**



Vincolo Disjoint e Complete

Una generalizzazione può essere **sia disjoint sia completa**, quindi ogni istanza della superclasse è anche istanza di **esattamente una sottoclasse** della generalizz.



In questo esempio si ha:

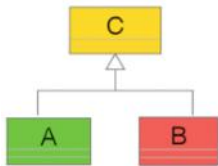
- Ogni istanza di Persona deve essere anche istanza di **esattamente una sottoclasse tra Uomo e Donna**
- Ogni istanza di Persona deve essere anche istanza di **esattamente una sottoclasse tra Studente e Lavoratore**
- Una istanza di Persona può essere istanza **sia di Uomo che di Studente**? Si
- Una istanza di Persona può essere istanza **né di Uomo né di Donna**? **Non più**
- Una istanza di Persona può essere istanza **sia di Uomo che di Donna**? **Non più**

Riepilogo

Generalizzazione Normale

Ereditarietà classica con la **relazione is-a** tra superclasse e sottoclassi.

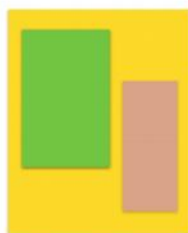
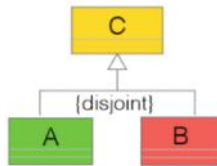
Il criterio della generalizz. è **opzionale**



Un istanza di C può essere anche istanza di A o di B (o entrambe) o niente

Generalizzazione Disjoint

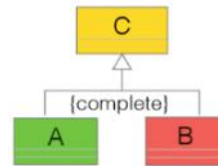
Il **vincolo disjoint** non permette ad un oggetto di **essere istanza di più sottoclassi** della stessa generalizzazione



Un istanza di A non può essere anche istanza di B

Generalizzazione Complete

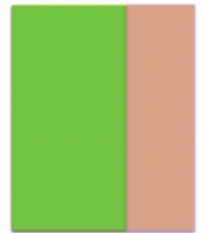
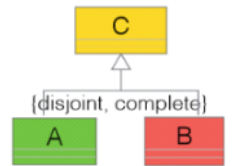
Il **vincolo complete** non permette ad un oggetto di **non essere istanza di nessuna sottoclasse** della stessa gener.



Un istanza di C deve essere per forza anche istanza di A o B (o entrambe)

Disjoint e Complete

Entrambi i vincoli impongono che ogni oggetto **sia istanza esattamente di una sola sottoclasse** della stessa generalizz.



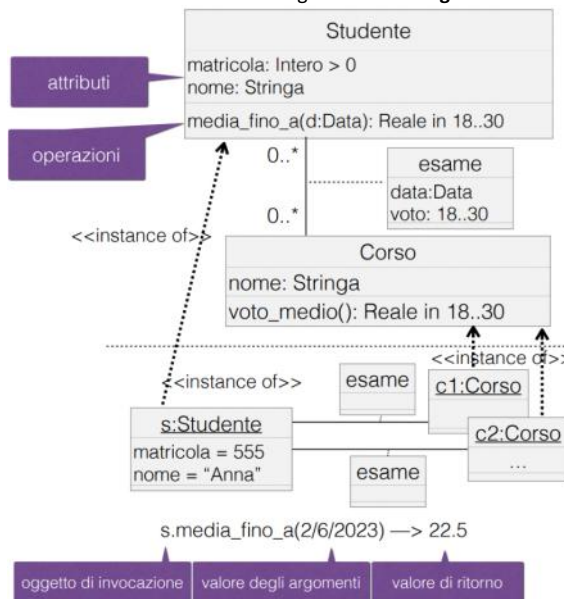
Una istanza di C deve essere per forza anche istanza di **solo una classe** tra A e B

Operazioni di Classe

venerdì 14 marzo 2025 11:02

UML, per le classi, può definire proprietà:

- **Statiche:** quelle viste in precedenza, dove i val. **cambiano** dopo la **modifica da** parte degli **utenti** del sistema
- **Dinamiche:** dove i val. vengono **calcolati ogni volta** che servono, a **partire** dai **valori** di altre proprietà



Operazioni di Classe

Una **op. della classe C** indica che su ogni oggetto di quella classe si può eseguire un calcolo per:

- **Calcolare** un val. a partire da altri val/operazioni
- **Cambiare lo stato dell'oggetto** (modificare le sue proprietà), **dei link** in cui è coinvolto e/o degli oggetti collegati a esso.

Sintassi

nome_ogg.nome_op(argomenti): tipo_ritorno

- **"argomenti"**: lista di elem. in forma **nome_argomento: tipo_argomento**
- **"tipo_ritorno"**: tipo del val. **restituito** dall'operazione

I tipi degli argomenti/ritorno possono essere tipi di **dato concettuali** o **classi del diagramma**.

L'op. di classe si può **invocare solo dall'oggetto** della classe.

Esempio:

s.media_fino_a(2/6/2023) -> 22.5

- **"s"** è l'oggetto della classe **Studente** che esegue la chiamata dell'operazione
- **"media_fino_a"** è l'operazione della classe
- **"2/6/2023"** è il val. di tipo **Data** **mandato** come **argomento** nell'operazione
- **"22.5"** è il val. di tipo **Reale** che viene **restituito** dall'operazione

L'op. di classe può essere chiamata anche da una **sottoclasse** della classe in cui vi è l'operazione

Specifiche delle Operazioni

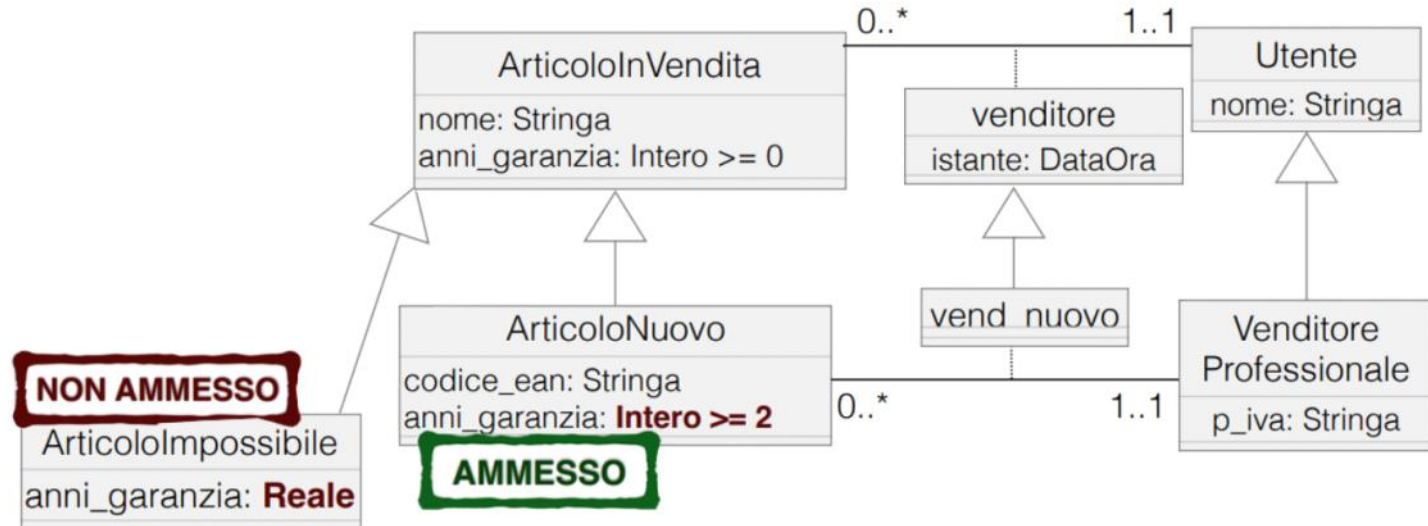
Il diagramma delle classi non definisce cosa calcolano/modificano le op, quindi ogni classe con op. andrà affiancata da un **documento di specifica** che entra nel dettaglio. (CREARE COLLEGAMENTO DOPO)

Specializzazione di Attributi

venerdì 14 marzo 2025 13:31

Una sottoclasse, oltre a proprietà aggiuntive, può anche **specializzare** le proprietà **ereditate** dalla superclasse, **restringendone il tipo** (lasciando lo stesso tipo di dato)
Esempio:

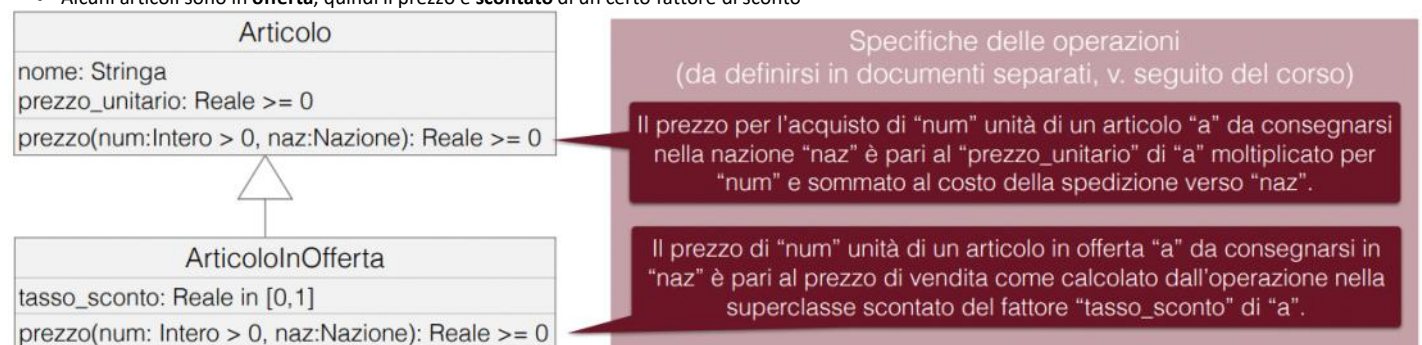
- Degli articoli in vendita si rappresenta il nome e gli anni di garanzia (anche zero)
- Alcuni articolo sono **nuovi** e per questi ci interessa l'EAN e che l'anno di garanzia deve essere **almeno due anni**
- Un utente può vendere **esattamente** un libro tra quelli in vendita
- Gli articoli nuovi li può vendere invece **solo** i venditori professionali
- ATTENZIONE:** l'assoc. **vend_nuovo** deve essere di tipo compatibile con il tipo dell'assoc. **venditore**



Anche le **op. di classe** possono essere oggetto di **specializzazioni nella sottoclasse**

Esempio:

- Di tutti gli articoli in vendita va calcolato il prezzo = prezzo unitario * num. pezzi richiesto + spese di spedizione (dipende dalla nazione dove viene consegnata)
- Alcuni articoli sono in **offerta**, quindi il prezzo è **scontato** di un certo fattore di sconto



Come per gli attributi, la segnatura dell'op. nella sottoclasse deve essere **compatibile** con quella dell'operazione definita nella superclasse, quindi:

- Stesso num. e tipo di argomenti**
- Il tipo di ritorno** nella sottoclasse è dello stesso tipo o un sotto-tipo di quello dell'operazione nella superclasse

Link utili Esame

venerdì 21 febbraio 2025 15:20

<https://bitbucket.org/mclab/workspace/projects/BD2>

Soluzioni esercitazioni di Analisi, SQL e alcuni progetti

Azienda 1

martedì 4 marzo 2025 15:02

Obiettivi

Si vuole sviluppare un sistema informativo per la gestione dei dati sul personale di una certa azienda costituita da diversi dipartimenti. Durante la fase di raccolta dei requisiti è stata prodotta la specifica dei requisiti mostrata di seguito.

Si chiede di iniziare la fase di **Analisi dei requisiti** ed in particolare di:

1. raffinare la specifica dei requisiti eliminando inconsistenze, omissioni o ridondanze e produrre un elenco numerato di requisiti il meno ambiguo possibile
2. produrre un diagramma UML delle classi concettuale che modelli i dati di interesse, utilizzando solo i costrutti di classe, associazione, attributo.

Requisiti

I dati di interesse per il sistema sono **impiegati**, **dipartimenti**, **direttori dei dipartimenti** e **progetti aziendali**.

Di ogni **impiegato** interessa conoscere il **nome**, il **cognome**, la **data di nascita** e lo **stipendio attuale**, il **dipartimento** (esattamente **uno**) al quale afferisce.

Di ogni **dipartimento** interessa conoscere il **nome**, il **numero di telefono del centralino**, e la **data di afferenza di ognuno degli impiegati che vi lavorano**.

Di ogni **dipartimento** interessa conoscere inoltre il **direttore**, che è **uno degli impiegati dell'azienda**.

Il **sistema** deve **permettere** di **rappresentare** i **progetti aziendali** nei quali sono coinvolti i **diversi impiegati**.

Di ogni **progetto** interessa il **nome** ed il **budget**.

Ogni **impiegato** può **partecipare** ad un **numero qualsiasi** di **progetti**.

Analisi dei Requisiti

Per ogni ruolo dobbiamo trovare i loro attributi.

Prova 1

Impiegato:

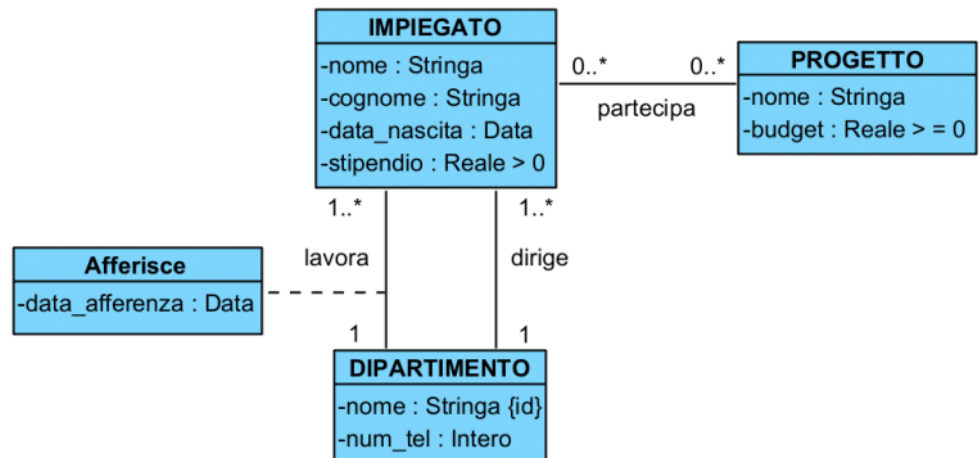
- Nome: Stringa
- Cognome: Stringa
- Data nascita: Data
- Stipendio attuale: Reale ≥ 0
- dipartimento (Uno): Dipartimento
 - Data di afferenza
- Progetti (infiniti): Progetto

Dipartimento:

- Nome: Stringa {univoco}
- num. tel: Intero
- Impiegati che ci lavorano: Impiegato
 - Data di afferenza
- Direttore (uno): Impiegato

Progetto:

- nome: Stringa
- budget: Reale ≥ 0
- Impiegati che ci lavorano (infiniti): Impiegati



Obiettivi

Si vuole sviluppare un sistema informativo per la gestione dei dati di una università. Durante la fase di raccolta dei requisiti è stata prodotta la seguente specifica dei requisiti. Si chiede di iniziare la fase di Analisi dei requisiti ed in particolare di:

1. raffinare la specifica dei requisiti eliminando inconsistenze, omissioni o ridondanze e produrre un elenco numerato di requisiti il meno ambiguo possibile
2. produrre un diagramma UML delle classi concettuale che modelli i dati di interesse, utilizzando solo i costrutti di classe, associazione, attributo.

Requisiti

I dati di interesse per il sistema sono studenti, facoltà, professori e corsi.

- Di ogni studente interessa conoscere il nome, il codice fiscale, il numero di matricola, la data di nascita, il luogo di nascita (città e regione), il corso di laurea a cui è iscritto (con l'anno di iscrizione), e gli insegnamenti di cui ha superato l'esame.
- Dei professori interessa il nome, la data di nascita, il codice fiscale, il luogo di nascita e gli insegnamenti erogati.
- Dei corsi di laurea interessa il nome e la o le facoltà di appartenenza.
- Di queste ultime interessa il nome.
- Di ogni insegnamento interessa il codice, il nome, il numero di ore di lezione, e i corsi di laurea a cui appartiene

Analisi dei Requisiti

Studente:

- nome: Stringa
- CF: Stringa (univoco)
- Matricola: Stringa
- Data_nascita: Data
- Luogo_nascita: (Città: Stringa, Regione: Stringa)
- Corso_laurea: → Corso (da 1 a inf) + anno iscrizione (association class)
- insegnamenti superati: Insegnamento (da 0 a inf)

Professori:

- nome: Stringa
- Data_nascita: Data
- CF: Stringa (univoco)
- Luogo_nascita: (Città: Stringa, Regione: Stringa)
- insegnamenti erogati: Insegnamento (da 1 a inf)

Studenti e Professori possono essere sottoclassi di Persona (con un disjoint così un prof non può essere studente), che ha:

- nome: Stringa
- CF: Stringa (univoco)
- Data_nascita: Data
- Luogo_nascita: (Città: Stringa, Regione: Stringa)

Corso:

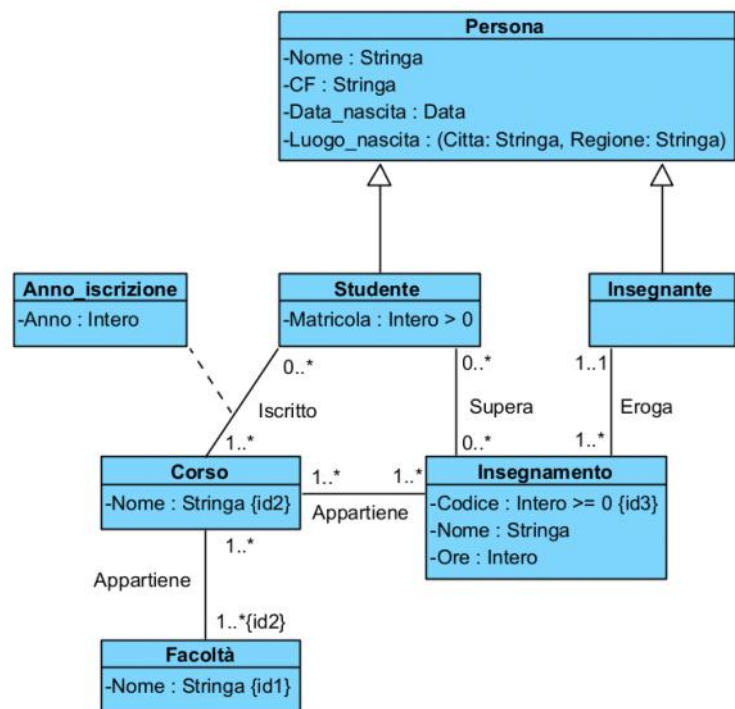
- nome: Stringa (univoco?)
- facoltà di appartenenza: Facoltà (da 1 a inf)
- Studente del corso: Studente (da 0 a inf)
- Insegnamenti: Insegnamento (da 1 a inf)

Facoltà:

- Nome (univoco)
- Corsi: Corso (da 1 a inf)

Insegnamenti:

- Codice: Intero (univoco)
- Nome: Stringa
- Ore_lezione: Intero
- Corso_appartenenza: Corso (da 1 a inf)
- Studenti nel corso: Studente (da 0 a inf)



Voli Aerei 1

venerdì 14 marzo 2025 17:29

Obiettivi

Si vuole sviluppare un sistema informativo per la gestione di dati relativi a voli aerei. Durante la fase di raccolta dei requisiti è stata prodotta la seguente specifica dei requisiti. Si chiede di iniziare la fase di Analisi dei requisiti ed in particolare di:

1. raffinare la specifica dei requisiti eliminando inconsistenze, omissioni o ridondanze e produrre un elenco numerato di requisiti il meno ambiguo possibile
2. produrre un diagramma UML delle classi concettuale che modelli i dati di interesse, utilizzando solo i costrutti di classe, associazione, attributo.

Requisiti

I dati di interesse per il sistema sono voli, compagnie aeree ed aeroporti.

- Dei voli interessa rappresentare codice, durata, compagnia aerea ed aeroporti di partenza e arrivo.
- Degli aeroporti interessa rappresentare codice, nome, città (con nome e numero di abitanti) e nazione.
- Delle compagnie aeree interessa rappresentare nome, anno di fondazione, e la città in cui ha sede la direzione.

Analisi dei Requisiti

Voli:

- Codice: Stringa (univoco)
- Durata: Reale
- Compagnia_aerea: Compagnia (una)
- Aereoporto_part: Aereoporto (uno)
- Aereoporto_arriv: Aereoporto (uno)

Aereoporto:

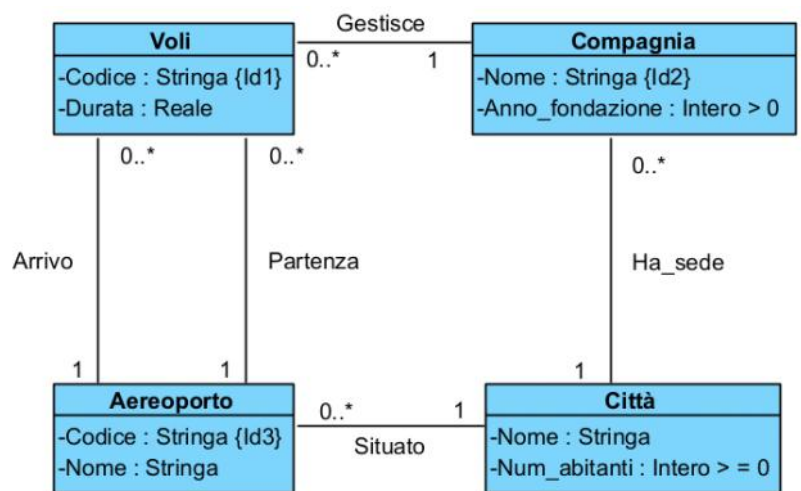
- Codice: Stringa (univoco)
- Nome: Stringa
- Città: Città (una)
- Nazione: Stringa (univoco) (non so se metterlo qua o in Città)
- Voli: Voli in uscita (da 0 a inf), voli in entrata (da 0 a inf)

Compagnia:

- Nome: Stringa (univoco)
- Anno_fondazione: Intero > 0
- Città_sede: Città (una)
- Gestisce piu voli: Voli (da 0 a inf)

Città:

- Nome: Stringa
- Num_abitanti: Intero > 0
- Aereoporti (da 0 a inf)
- Sedi_compagnie: Compagnia: (da 0 a inf)



Obiettivi

Si vuole progettare un sistema informativo per la gestione delle tabelle orarie relative al ruolo di docente universitario. Durante la fase di raccolta dei requisiti è stata prodotta la seguente specifica dei requisiti. Si chiede di iniziare la fase di Analisi Concettuale ed in particolare di :

1. raffinare la specifica dei requisiti eliminando inconsistenze, omissioni o ridondanze e produrre un elenco numerato di requisiti il meno ambiguo possibile
2. produrre un diagramma UML delle classi concettuale che modelli i dati di interesse, utilizzando solo i costrutti di classe, associazione, attributo
3. produrre la relativa specifica dei tipi di dato in caso si siano definiti nuovi tipi di dato concettuali.

Requisiti

I dati di interesse per il sistema sono i docenti universitari, i progetti di ricerca e le attività dei docenti.

- Di ogni docente interessa conoscere il nome, il cognome, la data di nascita, la matricola, la posizione universitaria (ricercatore, professore associato, professore ordinario) e i progetti ai quali partecipa.
- Dei progetti interessa il nome, un acronimo, la data di inizio, la data di fine e i docenti che vi partecipano.
- Un progetto è composto da molti Work Package (WP). Oltre al progetto a cui fa riferimento, del WP interessa sapere il nome, la data di inizio e la data di fine.
- Il sistema deve permettere ai docenti di registrare impegni di diverso tipo. Degli impegni interessa sapere il giorno in cui avvengono, la durata in ore e la tipologia di impegno con relativa motivazione.

Analisi dei Requisiti

Docente:

- Nome: Stringa
- Cognome: Stringa
- Data_nascita: Data
- Matricola: Intero (univoco)
- Posizione_Uni: Enum (Ricercatore, Prof associato, Prof ordinario)
- Progetti a cui partecipa: Progetto (da 0 a inf)
- Impegni (da 0 a inf)

Progetto:

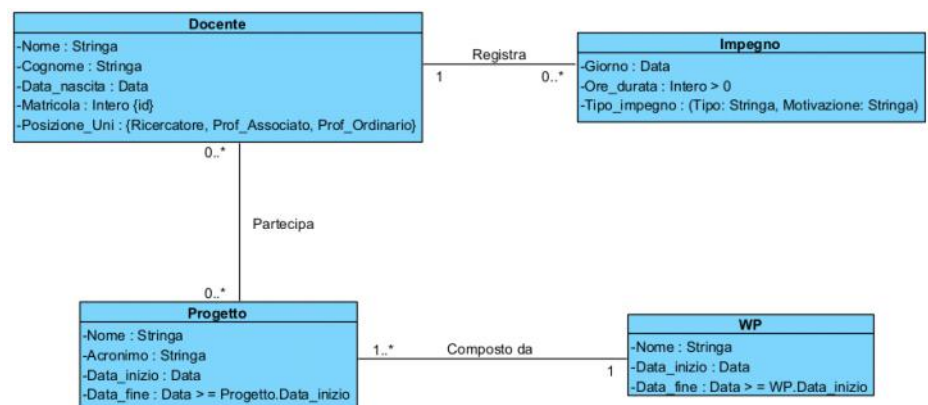
- Nome: Stringa
- Acronimo: Stringa
- Data_inizio: Data
- Data_fine: Data $\geq$ Data_inizio
- Docenti che vi partecipano: Docente (da 0 a inf)
- Work_package: WP (da 1 a inf)

WP:

- Nome: Stringa
- Data_inizio: Data
- Data_fine: Data $\geq$ data_inizio
- Progetti di cui fa parte: Progetto (uno)

Impegno:

- Giorno_avvenimento: Data
- Ore_durata: Intero > 0
- Tipo_impegno: (Tipo: Stringa, Motivazione: Stringa)
- Docente (uno)



Voli Aerei 2

venerdì 14 marzo 2025

17:32

Obiettivi

Si vuole sviluppare un sistema informativo per la gestione di dati relativi a voli aerei. Durante la fase di raccolta dei requisiti è stata prodotta la seguente specifica dei requisiti. Si chiede di iniziare la fase di Analisi Concettuale ed in particolare di:

1. raffinare la specifica dei requisiti eliminando inconsistenze, omissioni o ridondanze e produrre un elenco numerato di requisiti il meno ambiguo possibile
2. produrre un diagramma UML delle classi concettuale che modelli i dati di interesse, utilizzando solo i costrutti di classe, associazione, attributo, generalizzazione tra classi
3. produrre la relativa specifica dei tipi di dato in caso si siano definiti nuovi tipi di dato concettuali.

Requisiti

I dati di interesse per il sistema sono voli, compagnie aeree ed aeroporti.

- Dei voli interessa rappresentare codice, durata, compagnia aerea ed aeroporti di partenza e arrivo.
- Degli aeroporti interessa rappresentare codice, nome, città (con nome e numero di abitanti) e nazione.
- Delle compagnie aeree interessa rappresentare nome, anno di fondazione, e la città in cui ha sede la direzione.
- Un tipo particolare di voli sono voli charter. Questi possono prevedere tappe intermedie in aeroporti. Delle tappe intermedie di un volo charter interessa mantenere l'ordine con cui esse si susseguono (ad esempio, un certo volo che parte da "Milano Linate" e arriva a "Palermo Punta Raisi", prevede tappe intermedie prima nell'aeroporto di Bologna e poi in quello di Napoli). Dei voli charter interessa rappresentare anche il modello di velivolo usato.

Analisi dei Requisiti